

Frontend Developer Assessment (Next.js)

Preconditions

- All development should be completed using TypeScript.
- Please use Next.js version 15 or higher.
- Style the application with Tailwind CSS version 4 or above.
- Integrate components from [HeroUI](#).
- Ensure proper project setup, including:
 - Linter and code formatter configuration.
 - Pre-commit hooks.
- Implement robust UI handling for asynchronous states (e.g., loaders, skeletons).

API Resources:

- Base URL: <https://653fb0ea9e8bd3be29e10cd4.mockapi.io/api/v1>
- Documentation: [MockAPI Docs](#)

Main Task

- Implement support for both dark and light themes.
- Support two distinct account types:
 - **Member**: primary color #2AFC98
 - **Partner**: primary color #119DA4
- Ensure HeroUI components dynamically reflect the primary color based on the active user type.
- Develop a login form with OTP verification for both account types:
 - **Member**
 - Email: member@valid.email
 - Password: Member123!
 - OTP: 151588
 - **Partner**
 - Email: partner@valid.email
 - Password: Partner123!
 - OTP: 262699
- Store the authenticated user in `sessionStorage`.
- Implement infinite scroll pagination, search, and sorting:
 - Fetch data from the `/currencies` and `/balances` endpoints.
 - Map `currency_id` to its corresponding symbol and code.
 - Sorting, searching, and pagination should be handled via the API (see documentation).
- Create a dynamic page to display details by currency ID.

Requirements

- Submit your solution as a GitHub repository with a clear commit history and meaningful commit messages. Optionally, provide a live deployment link.
- Ensure the application is fully responsive and adheres to accessibility best practices (semantic HTML, keyboard navigation, appropriate ARIA attributes).
- Use modular, reusable components and maintain a clean, logical project structure.
- You may use React Context, Zustand, Redux, or another state management solution. Please justify your choice in the documentation.
- Implement robust error handling for all API interactions and user inputs.
- Provide a concise README with setup, usage, and testing instructions. Briefly explain your architectural decisions and any trade-offs made.

Bonus

- Write unit tests for custom hooks.
- Write functional tests for components, including handling both valid and invalid responses by mocking the HTTP client.
- Set up CI/CD pipelines (GitHub or GitLab) to run build processes and enforce code style checks.